

Assignment: Reliable UDP Communication in Mininet

Duration: 2 days | Topology: h1 -- s1 -- h2 | h1 = Server, h2 = Client

Objective

In this assignment, students will build a UDP client-server application in Mininet and add basic reliability using sequence numbers, ACKs, timeout, and retransmission. The task is designed to be doable within two days and easy to evaluate from code, screenshots, CSV results, and a short report.

1. Network Setup

Use the default Mininet topology:

```
h1 ---- s1 ---- h2
```

Start Mininet using:

```
sudo mn
```

Usually, h1 = 10.0.0.1 and h2 = 10.0.0.2. Verify the network before running the program:

```
nodes
net
pingall
```

2. Timeout Rule

Timeout is required because the client must decide how long it should wait for an ACK before assuming that the data packet or ACK was lost. UDP does not automatically retransmit lost packets. Therefore, in Stop-and-Wait communication, if the client does not receive the correct ACK within the timeout period, it retransmits the same packet.

Use the timeout value based on the last digit of your roll number. This makes each student's experiment slightly different and helps in individual evaluation.

Roll Number Last Digit	Timeout
0-1	0.5 s
2-3	0.7 s
4-5	1.0 s
6-7	1.2 s
8-9	1.5 s

3. Main Task: Reliable UDP using Stop-and-Wait

Write a UDP server on h1 and a UDP client on h2. The client must send 10 messages to the server.

Example messages:

```
Message 1 from h2
Message 2 from h2
...
Message 10 from h2
```

Each packet must contain a sequence number.

Packet format:

SEQ|MESSAGE

Example:

3|Message 3 from h2

The server must reply with an ACK.

ACK format:

ACK|SEQ

Example:

ACK|3

4. Program Requirements

Client Requirements	Server Requirements
1. Send one message at a time. 2. Wait for ACK. 3. Retransmit if ACK is not received before timeout. 4. Count total packets sent. 5. Count retransmissions. 6. Measure transfer time.	1. Receive messages. 2. Send ACKs. 3. Detect duplicate messages. 4. Count unique messages. 5. Count duplicate messages.

5. Experiment

Run the final program under the following three Mininet conditions. Run each condition once.

```
sudo mn --link tc,bw=5,delay=30ms,loss=0
sudo mn --link tc,bw=5,delay=30ms,loss=5
sudo mn --link tc,bw=5,delay=30ms,loss=10
```

6. Required Output

The client output must show:

```
TOTAL_MESSAGES=10
LOSS_PERCENT=__
TIMEOUT=__
TOTAL_PACKETS_SENT=__
TOTAL_RETRANSMISSIONS=__
TRANSFER_TIME_SECONDS=__
STATUS=SUCCESS
```

The server output must show:

```
TOTAL_UNIQUE_MESSAGES_RECEIVED=10
TOTAL_DUPLICATES_DETECTED=__
STATUS=SUCCESS
```

7. Submission Files

Submit one folder named ROLLNO_NAME_UDP_ASSIGNMENT. It must contain:

```
server.py
client.py
result_table.csv
screenshots/
report.pdf
```

The screenshots folder must contain:

```
nodes.png
net.png
pingall.png
server_output.png
client_output.png
```

8. Result Table

Submit result_table.csv with exactly these columns:

```
roll_no,name,loss_percent,timeout,total_messages,total_packets_sent,total_retransmissions,transfer_time_seconds,status
```

There must be exactly 3 data rows: loss = 0, loss = 5, and loss = 10.

9. Report

Submit a short report.pdf of around 2-3 pages. The report must include name, roll number, timeout used, Mininet topology, result table, and short answers to these questions:

1. Why is ACK needed?
2. Why is timeout needed?
3. Why can duplicate messages occur?
4. What happens when packet loss increases?
5. How is this method similar to TCP?
6. How is it different from TCP?

10. Marking Scheme

Component	Marks
Correct Mininet setup	10
UDP client-server program	15
Sequence number and ACK	15
Timeout and retransmission	20
Duplicate detection at server	10
Result table	10
Screenshots	10
Report	10
Total	100

11. Important Notes

- The program must run inside Mininet, not only on the normal terminal.
- The timeout must follow the roll-number rule.
- Do not change the CSV column names.
- Students may use resources, but they must understand and explain their own code.
- During evaluation, students may be asked to run the program and explain it.